ISA5810 DM2026 Lab 1 — Master Notebook Questions

Fill in this file as you work through DM2026-Lab1-Master.ipynb. Submit this completed file converted to PDF together with the rest of your Lab 1 work.

Question numbers 1–10 follow the same order as the notebook (top to bottom).

Student Information

Please complete the fields below. For **English Name** and **Chinese Name**, you may fill in **only one** if you do not have the other.

- **English Name:Rosso Lee**
- **Chinese Name:李中信**
- **Email:D11409105@mail.ntust.edu.tw**
- **Student ID:D11409105**
- **University:NTUST**

Instructions

These questions go with the Master Jupyter notebook (DM2026-Lab1-Master.ipynb). Each question is tied to a specific notebook section. Open the notebook, find **Question N (take home)**, and write your answer here.

We **strongly** suggest running the notebook to learn the lab, but the TAs will **not** audit whether you executed every cell to answer the questions in this document.

How to answer each question

1. Find the **notebook section** listed under the question (use the Table of Contents if needed).
2. Read the task; change parameters when asked (e.g. stop_words, thresholds, k, minSup).
3. Write a clear answer in the **Answer** space below each question.
4. Leave the question text unchanged; write only in the answer and disclosure fields.

Questions

Question 1: Data Filtering & Domain Relevance

Notebook: Section 3.2 — Question 1 (take home)

Task: Filter the dataframe **X** for the category **sci.med** and check its length.

Questions:

1. How many documents are in this specific category?
2. From a data mining perspective, why is isolating specific categories a crucial first step before applying algorithms like frequent pattern mining?

Answer:

There are 594 documents in the sci.med category. This result was obtained by filtering the DataFrame X using category_name == 'sci.med' and checking the number of matching records.

Isolating specific categories before frequent pattern mining is important because it ensures that the discovered patterns are relevant to the domain being analyzed. Mixing documents from unrelated categories may introduce irrelevant frequent terms and make the results harder to interpret. By focusing on a specific category, we can reduce noise and identify more meaningful domain-specific patterns.

AI used for this answer: Yes

If yes, how:

I used ChatGPT to help understand Pandas filtering, explain the importance of domain relevance in frequent pattern mining, and improve the clarity of my English explanation. The document count was verified by running the Python code in the notebook.

External references (links):

https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.loc.html

---

Question 2: Handling Missing Data

Notebook: Section 4.1 — Question 2 (take home) (after the **dropna()** demonstration)

Task: In Section 4.1, a dummy record with a missing value is injected and then removed using the **dropna()** function.

Question: If 40% of documents were missing the **category_name** label, why would simply dropping these rows (with **dropna()**) be a flawed data mining strategy? Name one objective alternative so the text data can still be utilized.

Answer:

Simply using dropna() to remove 40% of the documents would result in a substantial loss of potentially valuable text data. It could also introduce selection bias if the missing

labels are not randomly distributed, making the remaining dataset less representative of the original population.

One objective alternative is to retain the documents with missing category_name labels and use their text for unsupervised text mining, such as frequent pattern mining or clustering. These methods can identify patterns in the text without requiring category labels, allowing us to preserve and utilize the available information.

AI used for this answer: Yes

If yes, how:

I used ChatGPT to understand the limitations of deleting records with missing labels and to explore an alternative approach using unsupervised text mining. ChatGPT also helped organize and clarify my English explanation.

External references (links):

https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.dropna.html

---

Question 3: Data Sampling Risks

Notebook: Section 5.1 — Question 3 (take home)

Task: In Section 5.1, the code uses **X_sample = X.sample(n=1000)** to create a subset. Compare the bar chart of the original category distribution to the sampled distribution.

Questions:

1. Do the exact category proportions perfectly match the original?
2. What is the specific risk of simple random sampling on a highly imbalanced dataset?

Answer:

1.Do the exact category proportions perfectly match the original?

Answer:No. The category proportions in the sampled dataset do not perfectly match those in the original dataset. For example, comp.graphics increased from approximately 25.9% to 27.8%, while soc.religion.christian decreased from approximately 26.5% to 24.8%. However, the overall distributions remain relatively similar. These differences are caused by random sampling variation.

2.What is the specific risk of simple random sampling on a highly imbalanced dataset?

Answer: Simple random sampling may underrepresent minority classes in a highly imbalanced dataset. Some minority classes may have very few samples or may even be completely excluded from the sampled dataset. This can introduce sampling bias, reduce the model's ability to learn minority-class patterns, and lead to misleading evaluation results. Stratified sampling is a better alternative when maintaining the original class proportions is important, because it samples from each category separately.

AI used for this answer: Yes

If yes, how:

I used ChatGPT to help interpret the sampling results, understand the risk of minority-class underrepresentation, and improve the clarity of my English explanation. The numerical results were obtained from my Jupyter Notebook experiments.

External references (links):

https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.StratifiedShuffleSplit.html

---

Question 4: Text Vectorization and Matrix Sparsity

Notebook: Section 5.3 — Question 4 (take home)

Task: In Section 5.3, the **CountVectorizer** converts text into a document-term matrix using **stop_words='english'**. Change this parameter to **stop_words=None** and re-run the cell to check the shape of **X_counts**.

Questions:

1. Does the total number of features (columns) increase or decrease?
2. Since stopwords are highly common words that appear in almost all documents, what effect does adding them back have on the overall sparsity (the percentage of zeros) of the matrix?

Answer:

1. Does the total number of features (columns) increase or decrease?

The total number of features increases from 35,482 to 35,788 when changing stop_words='english' to stop_words=None. This represents an increase of 306 features because common English stopwords are no longer removed from the vocabulary.

2. What effect does adding stopwords back have on the overall sparsity of the matrix?

Adding stopwords decreases the overall matrix sparsity from 99.6772% to 99.5470%, a decrease of 0.1302 percentage points.

The number of non-zero elements increases from 258,503 to 365,886 because common stopwords appear in many documents. Although the number of features increases, the proportion of non-zero elements also increases, making the document-term matrix slightly denser.

AI used for this answer: Yes

If yes, how:

I used ChatGPT to understand how stopword filtering affects feature dimensionality and matrix sparsity. ChatGPT also helped me calculate and interpret the experimental results and improve the clarity of my English explanation. The numerical results were verified by executing Python code in Jupyter Notebook.

External references (links):

https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.CountVectorizer.html

---

Question 5: Feature Selection — Variance Thresholding

Notebook: Section 5.3.1 — Question 5 (take home)

Task: In the variance filtering section, re-run **variance_filter** using threshold values **threshold = 0.0, threshold = 0.01, and threshold = 0.05**. Record the number of features marked as **keep == True**.

Questions:

1. What happens to the total size of the vocabulary as the variance threshold increases from 0.0 to 0.05?
2. In terms of document frequency, what kinds of words have variance close to zero and are eliminated first?
3. Why is eliminating near-zero variance features considered a safe way to reduce dimensionality without harming model accuracy?

Answer:

1. What happens to the total size of the vocabulary as the variance threshold increases from 0.0 to 0.05?

As the variance threshold increases, the number of retained terms decreases significantly. At thresholds of 0.0, 0.01, and 0.05, the numbers of retained terms are 35,482, 5,089, and 1,491, respectively. At a threshold of 0.05, approximately 95.8% of the original vocabulary is removed.

2. In terms of document frequency, what kinds of words have variance close to zero and are eliminated first?

Words that appear in very few documents (low document frequency) usually have near-zero variance because their counts are zero in most documents. Words that appear in nearly every document with similar frequencies can also have low variance. These terms show little variation across documents and are eliminated first.

3. Why is eliminating near-zero variance features considered a safe way to reduce dimensionality without harming model accuracy?

Near-zero variance features provide little information for distinguishing documents because their values are almost constant. Removing them reduces dimensionality, computational cost, and potential noise while generally preserving useful features for classification. However, model accuracy is not guaranteed to remain unchanged because some rare terms may still be important for identifying specific document categories.

Threshold	Number of terms kept (keep == True)
0.0	35,482
0.01	5,089
0.05	1,491

AI used for this answer: Yes

If yes, how:

ChatGPT was used to explain variance-based feature selection, interpret the experimental results, and assist in drafting the English answers. All three threshold experiments (0.0, 0.01, and 0.05) were executed in JupyterLab, and the reported feature counts were obtained from the actual notebook outputs.

External references (links):

Scikit-learn—VarianceThreshold:https://scikit-learn.org/stable/modules/generated/sklearn.feature_selection.VarianceThreshold.html

Question 6: Feature Correlation — Linear (Pearson) vs. Monotonic (Spearman)

Notebook: Section 5.3.2 — Question 6 (take home)

Task: Run the correlation filter against the binary indicator for the **comp.graphics** class. Inspect the top 10 features sorted by **pearson_r** and compare them to the top 10 features sorted by **spearman_r**.

Questions:

1. Are the top-correlated words generic stop words or domain-specific terminology for graphics?
2. What is the mathematical difference in what **Pearson** correlation measures versus what **Spearman** rank correlation measures?
3. In what specific scenario would a word have a high **Spearman** rank correlation with a topic, but a significantly lower **Pearson** correlation?

Answer:

1. Are the top-correlated words generic stop words or domain-specific terminology for graphics?

The top-correlated words contain a mixture of general vocabulary and graphics-related terminology.

For Pearson correlation, the top 10 features include graphics-related terms such as *vga*, *video*, *card*, and *algorithm*, but also general words such as *thanks*, *looking*, and *hi*. The highest-ranked Pearson feature is *thanks* (r = 0.2778), which is not a graphics-specific term.

For Spearman correlation, the top features include more directly relevant terms such as *graphics* (r = 0.5211), *image* (r = 0.3086), and *3d* (r = 0.3060). Other terms, including *files*, *windows*, and *format*, are related to the broader computing environment.

Therefore, the selected features are not exclusively graphics terminology. Both methods identify a mixture of domain-specific and general terms, but Spearman places several clearly graphics-related terms near the top.

2. What is the mathematical difference in what Pearson correlation measures versus what Spearman rank correlation measures?

Pearson correlation measures the strength and direction of a **linear relationship** between two variables using their original numerical values. It is calculated from the covariance divided by the product of their standard deviations.

Spearman correlation measures the strength and direction of a **monotonic relationship** by converting the observations into ranks and calculating the Pearson correlation of those ranks.

A monotonic relationship means that as one variable increases, the other generally increases or decreases consistently, even when the relationship is not linear.

Therefore, Pearson is sensitive to the magnitude of numerical values and extreme observations, while Spearman focuses on their relative ordering.

3. In what specific scenario would a word have a high Spearman rank correlation with a topic, but a significantly lower Pearson correlation?

One possible scenario occurs when a word is consistently associated with a topic, but its frequency varies greatly across documents.

For example, suppose the word *graphics* appears in many graphics-related documents but rarely appears in other categories. However, a few graphics-related documents repeat the word an unusually large number of times.

The word's occurrence pattern may provide a strong monotonic association with the topic, while the extreme frequency values can weaken the linear relationship measured by Pearson.

In my experiment, *graphics* had a Spearman correlation of 0.5211 but a Pearson correlation of only 0.1779. This illustrates the difference between the two measures, although additional examination of document-level word frequencies would be required to confirm the exact cause.

AI used for this answer: Yes

If yes, how:

ChatGPT was used to explain Pearson and Spearman correlation, interpret the experimental results, compare the selected features, and assist in drafting the English answers. I executed the correlation experiments in JupyterLab using the comp.graphics category and verified the reported results.

External references (links):

- https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.pearsonr.html
- https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.spearmanr.html

Question 7: Frequent Pattern Mining — Algorithm & Parameter Dynamics

Notebook: Section 5.4.2 — Question 7 (take home)

Task: In the Frequent Pattern Mining section, run **Top-K FAE** with **k = 100 and k = 500**. Then, run **MaxFPGrowth** using minimum support thresholds **minSup=9 and minSup= 3**.

Questions:

1. How does decreasing **minSup** (from 9 to 3) in **MaxFPGrowth** affect the execution runtime and the total number of discovered patterns?
2. How does the parameter k in **Top-K FAE** differ fundamentally from **minSup** in **MaxFPGrowth** when deciding which patterns to keep?
3. Why are "maximal" frequent patterns structurally more concise than standard frequent patterns?

Table 1. Comparison of Top-K FAE and MaxFPGrowth Results

Algorithm	Dataset	Parameter	Runtime (seconds)	Total Patterns
Top-K FAE	alt_atheism	k = 100	1.3233	100
Top-K FAE	alt_atheism	k = 500	0.2758	500
MaxFPGrowth	alt_atheism	minSup = 9	0.1732	710
MaxFPGrowth	alt_atheism	minSup = 3	3.1655	9,547

Key Observations:

- **Top-K FAE:** Increasing k from 100 to 500 increased the number of returned patterns from 100 to 500.
- **MaxFPGrowth:** Decreasing minSup from 9 to 3 increased the number of discovered maximal frequent patterns by approximately **13.45 times** and the runtime by approximately **18.28 times**.
- **Runtime interpretation:** The measured runtime for Top-K FAE was shorter at k = 500 than at k = 100. However, these were single-run measurements, so the difference does not establish a general performance trend.

Note: All experiments used the same term-frequency-based transactional database for the alt_atheism category. Runtime measurements may vary depending on system conditions.

Answer (1) — Runtime and pattern count when lowering minSup:

When the minimum support threshold decreased from 9 to 3, the number of maximal frequent patterns increased from **710 to 9,547**, approximately **13.45 times** as many patterns. The execution runtime also increased from **0.1732 seconds to 3.1655 seconds**, approximately **18.28 times** longer.This occurs because lowering minSup allows more low-frequency itemsets to qualify as frequent. Consequently, MaxFPGrowth must explore a larger search space and process more item combinations.The experimental results demonstrate that a lower minimum support threshold can significantly increase both the number of discovered patterns and the computational cost.

Answer (2) — Top-K k vs MaxFPGrowth minSup:

The fundamental difference is that **Top-K FAE uses a ranking-based selection criterion**, whereas **MaxFPGrowth uses a minimum support threshold**.In Top-K FAE, the parameter k specifies the number of top-ranked patterns to retrieve. In my experiment, increasing k from 100 to 500 increased the number of returned patterns from 100 to 500. The lowest support among the returned patterns decreased from 21 to 16.In contrast, minSup in MaxFPGrowth specifies the minimum number of transactions in which an itemset must appear to qualify as frequent. The number of returned maximal patterns is determined by the data and the threshold, rather than being fixed in advance.Therefore, k controls how many top-ranked patterns are retained, while minSup controls the minimum frequency required for patterns to be considered frequent.

Answer (3) — Maximal vs standard frequent patterns:

A **maximal frequent pattern** is a frequent itemset that has no frequent proper superset.For example, suppose {A, B, C} is frequent and no larger itemset containing it is frequent. Then {A, B, C} is a maximal frequent pattern.Although its subsets, such as {A}, {A, B}, and {B, C}, are also frequent, they do not need to be reported separately when mining maximal frequent patterns.As a result, maximal frequent pattern mining reduces redundant subsets and provides a more compact representation of frequent itemsets than standard frequent pattern mining.However, this compact representation does not preserve the individual support counts of every frequent subset. Therefore, maximal frequent patterns offer better structural conciseness at the cost of some detailed frequency information.

AI used for this answer: Yes

If yes, how:

I used ChatGPT to help explain the concepts of Top-K FAE, MaxFPGrowth, minimum support, and maximal frequent patterns. ChatGPT also assisted with interpreting the experimental results, calculating the relative changes in runtime and pattern counts, and organizing the answers in English.All four experiments were executed by me in JupyterLab using the course notebook and the alt_atheism dataset. The reported runtimes and pattern counts are based on my actual execution results.

External references (links):

- PAMI documentation: https://www.aidoczh.com/pami/
- Han, J., Pei, J., & Tong, H. (2022). *Data Mining: Concepts and Techniques* (4th ed.). Morgan Kaufmann.

Question 8: Dimensionality Reduction — Motivation

Notebook: Section 5.5 — Question 8 (take home) (before PCA/t-SNE code)

Task: After text vectorization, the **X_counts** matrix creates tens of thousands of distinct features (words).

Questions:

1. If we want to visually map how these documents group by category on a scatter plot, why is it mathematically and visually impossible to plot the raw **X_counts** matrix directly?
2. What specific category of data mining algorithms (referenced in the helper files) must be applied first?

Answer:

After text vectorization, the X_counts matrix contains 2,257 documents and 35,482 word features. Each document is represented as a point in a 35,482-dimensional feature space.

1. Why can't we directly visualize the raw X_counts matrix in a scatter plot?

A standard 2D scatter plot has only two coordinate axes (X and Y), while a 3D scatter plot has three axes (X, Y, and Z). However, each document in X_counts contains 35,482 feature dimensions.Therefore, we cannot directly represent all 35,482 dimensions in a conventional 2D or 3D scatter plot without reducing or selecting dimensions. Plotting only two original word features would discard most of the information and might not reveal meaningful relationships between document categories. Furthermore, high-dimensional text data is typically sparse, making it difficult to understand document similarities and category groupings through direct visualization.

2. What category of data mining algorithms must be applied first?

We need to apply **Dimensionality Reduction algorithms** to transform the high-dimensional document vectors into a lower-dimensional representation, usually two or three dimensions.

Section 5.5 of the notebook introduces three techniques:

- **PCA (Principal Component Analysis):** A linear method that projects data onto directions of maximum variance.
- **t-SNE (t-Distributed Stochastic Neighbor Embedding):** A non-linear method that emphasizes preserving local relationships between data points.
- **UMAP (Uniform Manifold Approximation and Projection):** A non-linear method designed to preserve important neighborhood and structural relationships.

These methods allow us to represent each document as a point in a 2D or 3D scatter plot and visually explore how documents from the four categories are grouped.

AI used for this answer: Yes

If yes, how:

I used ChatGPT to help explain why high-dimensional text vectors cannot be directly visualized in conventional 2D/3D scatter plots, clarify the purpose of dimensionality reduction, and organize my explanation in English. The matrix dimensions were obtained from my previous notebook execution, and the dimensionality reduction methods were referenced from Section 5.5 of the course notebook.

External references (links):

https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.TruncatedSVD.html

---

Question 9: Dimensionality Reduction — Linear (PCA) vs. Non-linear Manifolds (t-SNE/UMAP)

Notebook: Section 5.5 — Question 9 (take home) (after 2D/3D plots)

Task: In the 2D/3D Dimensionality Reduction section, compare the scatter plot projection generated by **PCA** against that generated by **t-SNE** or **UMAP**.

Questions:

1. Which method **(PCA or t-SNE/UMAP**) shows distinct, separated cluster islands for the 4 document categories, and which shows heavy overlap around the center?
2. Why does **PCA** struggle to separate high-dimensional text categories compared to **t-SNE** or **UMAP**?
3. Why must data analysts be careful not to over-interpret the physical distance between separate clusters in a **t-SNE** plot?

Answer:

1. Which method shows distinct cluster islands, and which shows heavy overlap around the center?

In my experiment, PCA showed heavy overlap among the four document categories, with most data points concentrated near the origin. Several extreme outliers also stretched the coordinate axes, making the main distribution appear compressed. In contrast, t-SNE produced more distinguishable local clusters. For example, the soc.religion.christian documents were concentrated mainly in the upper region, while some sci.med documents formed separate clusters in the lower region. UMAP also revealed some isolated clusters, particularly for a subset of sci.med documents. However, substantial overlap remained in the main distribution. Therefore, **t-SNE and UMAP revealed clearer local cluster structures than PCA**, although none of the three methods completely separated all four document categories.

2. Why does PCA struggle to separate high-dimensional text categories compared to t-SNE or UMAP?

PCA is a linear dimensionality reduction technique that finds directions of maximum variance in the data. However, directions with the greatest variance do not necessarily correspond to differences between document categories. High-dimensional text data is sparse, and relationships between documents may be complex and non-linear. Consequently, PCA can preserve dominant variations in word frequencies without clearly separating semantic categories. In contrast, t-SNE and UMAP are non-linear dimensionality reduction techniques that emphasize neighborhood relationships. They can reveal local structures that are difficult to capture using a linear projection. This explains why t-SNE and UMAP produced more visible category-related clusters in my experiment.

3. Why should analysts avoid over-interpreting physical distances between clusters in a t-SNE plot?

t-SNE primarily aims to preserve local neighborhood relationships rather than exact global distances. As a result, the distance between two separated clusters in a t-SNE visualization does not necessarily represent their true distance or semantic dissimilarity in the original high-dimensional feature space. Additionally, cluster positions, shapes, and apparent separation can change depending on parameters such as perplexity, initialization, and random seed. Therefore, t-SNE should be used mainly for exploratory visualization of local patterns, rather than as definitive evidence of the absolute similarity or dissimilarity between document categories.

Figure 1:PCA: Heavy category overlap near the origin, with extreme outliers.

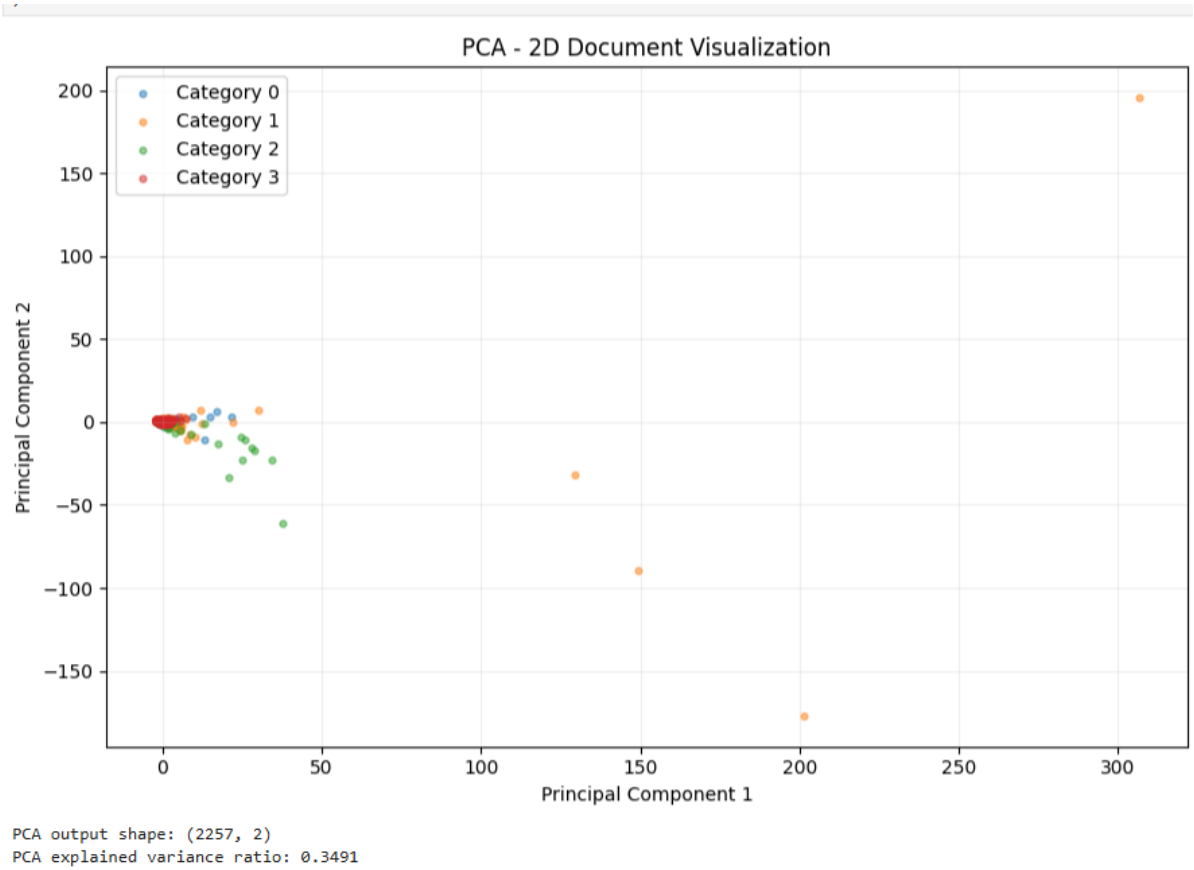
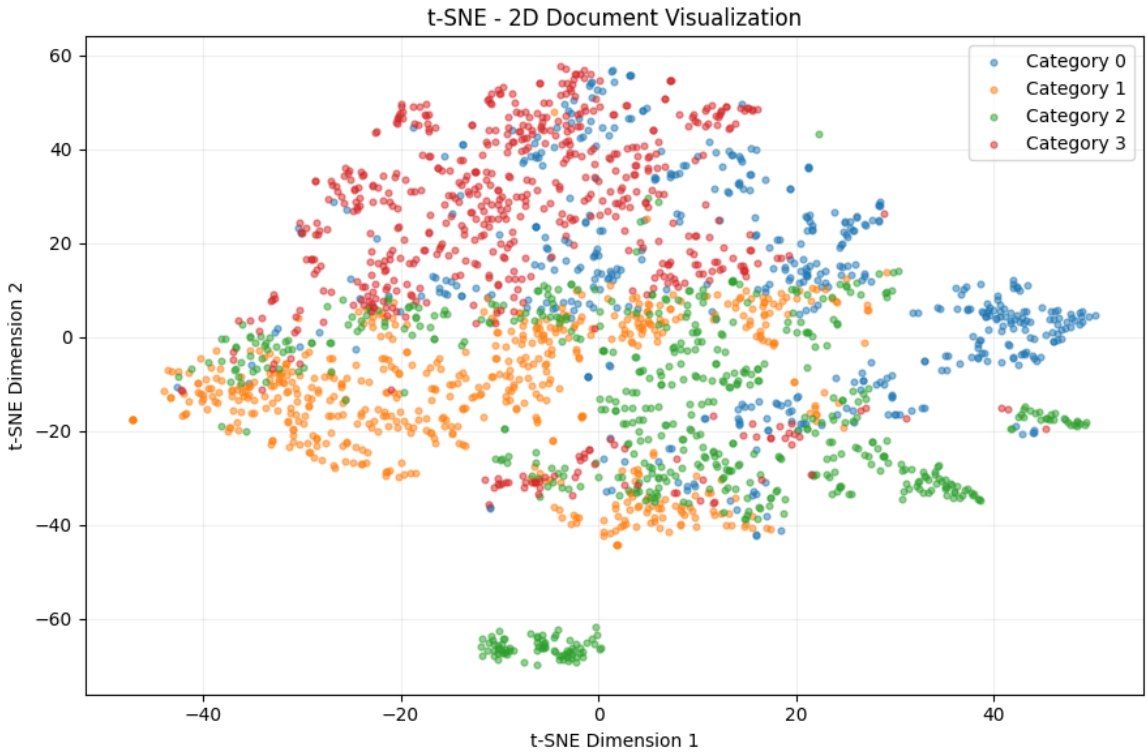
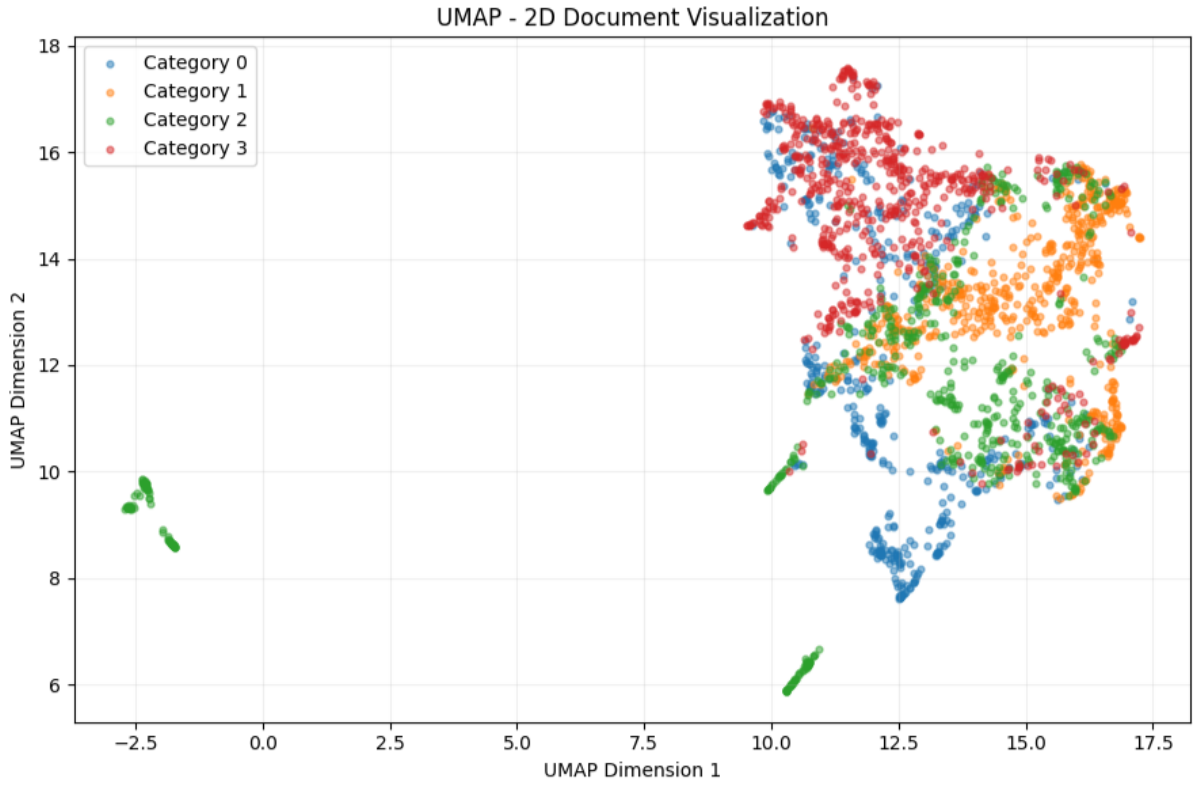


Figure 2:t-SNE: More visible local clusters, although category overlap remains.




```
t-SNE output shape: (2257, 2)
t-SNE KL divergence: 1.3486
```

Figure 3:UMAP: Some isolated clusters, but substantial overlap in the main distribution.




```
UMAP output shape: (2257, 2)
```

All visualizations use 2,257 documents from four categories, with TruncatedSVD reducing 35,482 original features to 50 dimensions before 2D projection.

AI used for this answer: Yes

If yes, how: I used ChatGPT to help explain PCA, t-SNE, and UMAP, develop memory-efficient dimensionality reduction code, interpret the generated scatter plots, and organize the answers in English. I executed the experiments in JupyterLab using the four-category dataset containing 2,257 documents. To reduce memory usage, I first applied TruncatedSVD to reduce the original 35,482 text features to 50 dimensions, then generated 2D PCA, t-SNE, and UMAP visualizations. The observations and comparisons are based on my actual experimental plots.

External references (links):

https://scikit-learn.org/stable/modules/generated/sklearn.manifold.TSNE.html

---

Question 10: One-Hot Binarization

Notebook: Section 5.6 — Question 10 (take home)

Task: In Section 3.1, examine **X['category']** (integers 0, 1, 2, 3). Then look at the output of **LabelBinarizer** in Section 5.6, where each category is converted into a binary vector (e.g., [1, 0, 0, 0]).

Questions:

1. How many binary columns are created to represent the 4 categories?
2. If an algorithm (like linear regression or KNN) is given the raw integer column **(0, 1, 2, 3)** as input, what false mathematical assumption does it make about the categories?
3. How does one-hot binarization eliminate this false assumption?

Answer:

1. How many binary columns are created to represent the four categories?

The LabelBinarizer creates **four binary columns**, one for each document category.

The experimental results are shown below:

Original Category	One-Hot Encoded Vector
0	[1, 0, 0, 0]
1	[0, 1, 0, 0]
2	[0, 0, 1, 0]
3	[0, 0, 0, 1]

Each category is represented by a unique binary vector containing exactly one value of 1 and three values of 0.

2. What false mathematical assumption is made when raw integer categories (0, 1, 2, 3) are used as input?

When raw category integers are used as numerical input features, algorithms such as linear regression or KNN may incorrectly assume that the categories have a meaningful numerical order and measurable distances. For example, Category 3 appears greater than Category 1, and the numerical distance between Category 0 and Category 3 is three times the distance between Category 0 and Category 1. However, these numbers are only category identifiers. They do not represent magnitude, ranking, or semantic similarity between document categories. Therefore, treating them as continuous numerical features introduces artificial ordinal and distance relationships.

3. How does one-hot binarization eliminate this false assumption?

One-hot binarization replaces each integer category with a separate binary indicator vector. Each category is represented independently, without implying that one category is greater or smaller than another. For example, Category 0 is encoded as [1, 0, 0, 0], while Category 3 is encoded as [0, 0, 0, 1]. The Euclidean distance between any two distinct one-hot vectors is the same, preventing the original integer labels from creating artificial differences in numerical distance. As a result, one-hot encoding allows algorithms to treat the categories as nominal rather than ordinal variables.

AI used for this answer: Yes

If yes, how: I used ChatGPT to help explain the mathematical assumptions associated with integer category encoding, clarify how one-hot binarization works, and organize my answers in English. I independently executed the LabelBinarizer experiment in JupyterLab using the four document categories (0, 1, 2, 3). The reported encoding results were verified against the actual notebook output.

External references (links):

https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.LabelBinarizer.html

---